

FASE 2 — Identidad de la persona + OTP (módulo intercambiable)

Objetivo: que una persona pueda identificarse/registrarse por **celular verificado (OTP)** de forma segura, con un módulo de OTP **intercambiable** (simulado hoy; Twilio/WhatsApp mañana). No se envían SMS/WhatsApp reales en esta fase.

1. Piezas creadas

Utilidades de servidor

- `src/lib/supabase-server.ts`
- `getServiceRoleClient()` → cliente con `SUPABASE_SERVICE_ROLE_KEY` (bypassa RLS). Solo servidor.
- `getServerSupabase()` → cliente ligado a cookies (respeta RLS) para el Lead Center.
- **Sin claves hardcodeadas:** todo por variables de entorno.
- `src/lib/phone.ts` — normalización a **E.164** sin dependencias externas (CO por defecto, más MX/PE/CL/EC/VE/PA/US). Devuelve `{ e164, pais, valido, motivo }` y `enmascararCelular()`.

Módulo OTP intercambiable (`src/lib/otp/`)

- `OtpProvider.ts` — contrato (`requestCode` , `verifyCode`) + tipos.
- `SimulatedOtpProvider.ts` — implementación de esta fase:
- Código de **6 dígitos**; se persiste **solo el hash bcrypt** (`bcryptjs`), nunca el código en claro.
- **Caducidad** 5 min, **máximo 5 intentos**, **rate limiting** (1/min y 5/hora por celular).
- Invalida retos pendientes previos al pedir uno nuevo.
- Registra la intención de envío en `comunicaciones_transaccionales` con estado **pendiente** (no envía nada real).
- Devuelve `codigoDemo` **solo** en modo simulado, para que la UI lo muestre bajo aviso.
- `TwilioOtpProvider.ts` / `WhatsAppOtpProvider.ts` — **stubs** listos para implementar (ver `MIGRATION_TO_REAL_OTP.md`). Reutilizan `desafios_otp` sin cambios.
- `index.ts` — `getOtpProvider()` selecciona por `OTP_PROVIDER` (default `simulated`).

API (Route Handlers, runtime nodejs)

- `POST /api/otp/request` — normaliza celular, aplica rate limit, genera reto. Expone `codigoDemo` solo si el proveedor es simulado.
- `POST /api/otp/verify` — verifica código (6 dígitos), controla caducidad/intentos, marca el reto **verificado** y devuelve `desafioId` / `personaId` / `visitanteId`.

Componentes UI (mobile-first, `components/leadcenter/`)

- `OtpInput.tsx` — 6 casillas con autofocus, pegado y teclado numérico.
- `VerificacionCelularModal.tsx` — flujo pedir→verificar; muestra “Código de demostración” solo en modo simulado.
- `RegistroPersonaModal.tsx` — captura nombre/correo + verificación de celular (propósito `registro`).
- `LoginCelularModal.tsx` — inicio de sesión por celular (propósito `login`).

2. Seguridad del OTP

- El código **nunca** se guarda ni se loguea en claro (solo `bcrypt` hash).
- La tabla `desafios_otp` tiene RLS y **no** es accesible por clientes anónimos/autenticados normales; toda la operación pasa por el servidor con service role.
- El `codigoDemo` viaja al cliente **solo** en modo simulado y se muestra con aviso explícito de demostración.
- Caducidad + intentos + rate limiting mitigan fuerza bruta y enumeración.

3. Reconciliación de identidad (visitante ↔ persona)

- El explorador ya crea un `visitante_id` anónimo (`src/lib/visitor.ts` , intacto).
- Al verificar el celular se propaga `visitante_id` al reto OTP; el flujo de conversión (Fase 3) vincula ese visitante con la `persona` (por `personas.visitante_id` y `personas.celular_e164`), sin fusionar registros automáticamente ante colisión.

4. Variables de entorno requeridas (no incluidas aquí)

```

NEXT_PUBLIC_SUPABASE_URL=...
NEXT_PUBLIC_SUPABASE_ANON_KEY=...
SUPABASE_SERVICE_ROLE_KEY=...      # solo servidor
OTP_PROVIDER=simulated              # simulated | twilio | whatsapp

```

5. Estado de verificación

- **Código escrito y con tipado consistente.** Se instaló `bcryptjs` + `@types/bcryptjs` .
- **NO ejecutado en runtime real:** sin credenciales Supabase en este entorno, no se puede llamar a las rutas contra una BD real ni completar un OTP de extremo a extremo. Queda **preparado**; la verificación E2E se hará al desplegar con variables de entorno.
- La compilación del proyecto se valida en la Fase 6 (`npm run build`).

6. Riesgos

Riesgo	Mitigación
Columnas de <code>comunicaciones_transaccionales</code> distintas	El insert va dentro de try/catch y no bloquea el OTP; se ajustará al aplicar.
Normalización E.164 limitada a países objetivo	Cubre LatAm principal + US; ampliable en <code>phone.ts</code> sin tocar el flujo.
Enumeración de celulares	<code>desafios_otp</code> sin lectura pública; respuestas de error genéricas.